Министерство образования Республики Беларусь
Учреждение образования
«БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ИНФОРМАТИКИ И РАДИОЭЛЕКТРОНИКИ»

Кафедра информатики

ОТЧЕТ
О ЛАБОРАТОРНОЙ РАБОТЕ № 4
ВЗАИМОДЕЙСТВИЕ ПРОЦЕССОВ(ПОТОКОВ): ВЗАИМНОЕ ИСКЛЮЧЕНИЕ И СИНХРОНИЗАЦИЯ

по дисциплине «Операционные среды и системное программирование»

Выполнил ст. гр. 453504 Л. Д. Масалова

Проверил старший препод. каф. Информатики А. Д. Сыс

Минск 2026

1 ЦЕЛЬ РАБОТЫ

Изучение механизмов взаимного исключения и синхронизации в операционной системе Windows, приобретение практических навыков организации согласованной параллельной работы процессов и потоков.

В ходе выполнения лабораторной работы необходимо:

– изучить основные проблемы конкурентного доступа к общим ресурсам;

– освоить системные объекты и функции Windows API для синхронизации;

– реализовать модель взаимодействия процессов или потоков;

– обеспечить корректный доступ к разделяемым ресурсам;

– исследовать возникновение и предотвращение состояний гонки, взаимных блокировок и длительного ожидания;

– оценить влияние механизмов синхронизации на производительность;

– приобрести навыки экспериментального исследования многопоточных программ;

– закрепить навыки применения системы контроля версий Git.

2 ЗАДАНИЕ

Вариант 15. Групповое изменение записей.

Создать массив из 100 целых чисел.

Писатели выполняют операции переноса значения между двумя элементами массива.

Каждый перенос должен сохранять общую сумму.

Сравнить:

1 Исключительную блокировку на всю группу переносов.

2 Блокировку отдельно на каждый перенос.

Исследовать размеры групп 5 и 20 операций.

Сравнить время ожидания читателей и пропускную способность.

Индивидуализация задания (STUDENT_ID = 72):

В соответствии с требованиями раздела индивидуализации параметры программы рассчитываются по формулам:

Базовое зерно генератора псевдослучайных чисел: SEED=1000+STUDENT_ID×97=1000+72×97=7984.

Количество операций на каждый рабочий поток: OPERATIONS=200+STUDENT_ID×5=200+72×5=560 операций.

Базовая сумма массива: INITIAL_SUM=100×10=1000.

3 РЕЗУЛЬТАТ РАБОТЫ

3.1 Описание предметной области и задачи

Предметной областью данной работы является многопоточная синхронизация в операционной системе Windows при конкурентном доступе к разделяемым структурам данных в оперативной памяти. Рассматривается фундаментальная модель параллельного программирования «Писатели–Читатели» (Readers–Writers Problem), в которой потоки разделяются на две категории с асимметричными требованиями к изоляции. Потоки-читатели выполняют только инспекцию данных, поэтому они могут работать в памяти одновременно без риска взаимного повреждения состояния. Напротив, потоки-писатели осуществляют модификацию данных, что требует строгого монопольного доступа для предотвращения состояния гонки (Race Condition).

В рамках варианта №15 исследуется задача группового переноса числовых значений внутри общего целочисленного массива из 100 элементов. Каждый элементарный перенос представляет собой транзакцию, уменьшающую значение в одной случайной ячейке и увеличивающую значение в другой на ту же величину, что сохраняет базовый инвариант общей суммы массива, равный 1000. Главная сложность задачи заключается в исследовании компромисса между гранулярностью блокировок: захват системного замка на всю группу из 5 или 20 операций обеспечивает максимальную скорость работы самого писателя, но заставляет читателей длительное время простаивать в очереди, в то время как поодиночная блокировка каждого переноса снижает время ожидания читателей за счет появления окон доступности ресурса, однако увеличивает системные накладные расходы на постоянный захват и освобождение примитива синхронизации.

3.2 Алгоритм функционирования приложения

Архитектура приложения построена на базе легковесного системного примитива синхронизации пользовательского режима SRWLOCK (Slim Reader/Writer Lock). На этапе инициализации программа выделяет массив из 100 элементов, заполняет каждую ячейку значением 10 и инициализирует замок вызовом InitializeSRWLock(). Далее главный поток посредством вызовов CreateThread() порождает пул из трех потоков-читателей и двух потоков-писателей, передавая каждому индивидуальную структуру параметров с настройками текущего эксперимента.

Функционирование читателей заключается в циклическом считывании массива: перед входом в критическую секцию запускается высокоточный аппаратный таймер QueryPerformanceCounter(), после чего поток запрашивает совместный доступ через системную функцию

AcquireSRWLockShared(). Сразу после захвата замка таймер останавливается, фиксируя точное время нахождения читателя в очереди ожидания в микросекундах. Находясь под защитой замка, читатель вычисляет сумму всех элементов массива и выполняет валидацию инварианта: если сумма строго равна 1000, состояние признается согласованным, в противном случае фиксируется сбой синхронизации. Завершив расчет, поток освобождает замок вызовом ReleaseSRWLockShared().

Потоки-писатели реализуют две исследуемые стратегии конкурентной модификации. При групповой блокировке писатель однократно захватывает монопольный доступ вызовом AcquireSRWLockExclusive(), выполняет подряд всю серию переносов (5 или 20 транзакций) с использованием независимого генератора псевдослучайных чисел std::mt19937, инициализированного индивидуальным зерном 7984, после чего освобождает ресурс через ReleaseSRWLockExclusive(). При поодиночной блокировке писатель выполняет парные вызовы захвата и освобождения эксклюзивного замка отдельно для каждого единичного переноса, предоставляя читателям возможность осуществить чтение между соседними транзакциями.

Главный поток координирует завершение работы всех участников конвейера с помощью барьерной синхронизации WaitForMultipleObjects(), закрывает дескрипторы завершившихся потоков функцией CloseHandle(), производит статистическую обработку зафиксированных временных задержек и формирует итоговые показатели пропускной способности системы. Схема взаимодействия потоков читателей и писателей с разделяемым массивом представлена на рисунке 1.

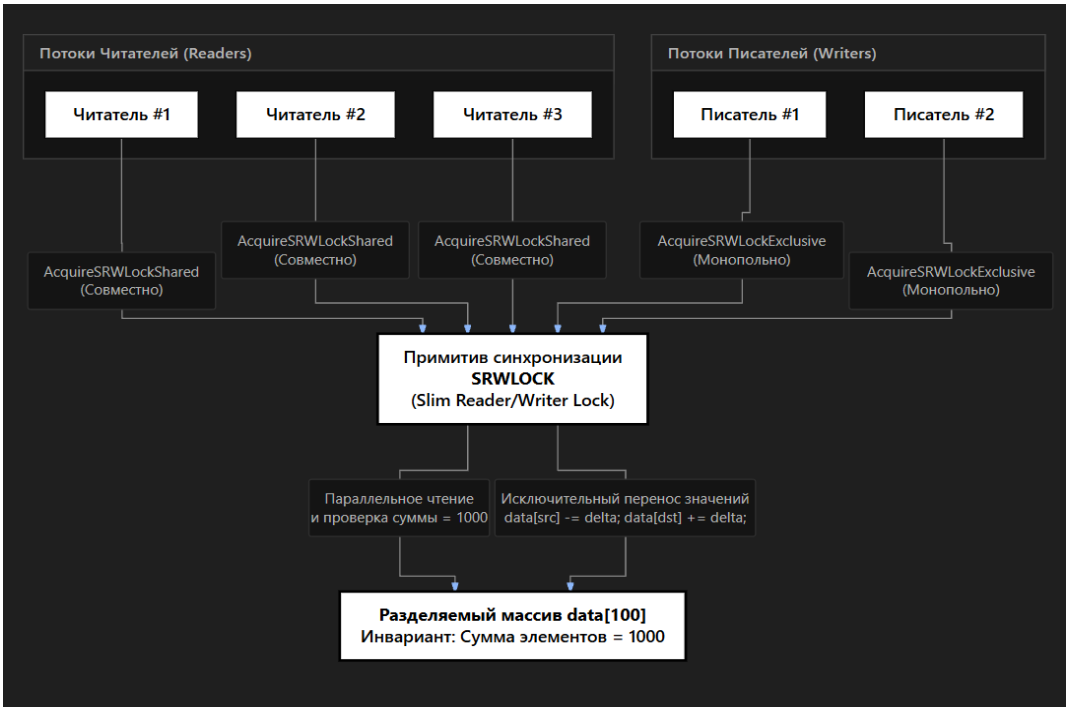


Рисунок 1 – Схема синхронизации читателей и писателей через SRWLock

3.3 Перечень использованных функций Windows API

В процессе реализации программы были задействованы следующие системные вызовы Windows API:

– InitializeSRWLock() — инициализация объекта тонкого замка чтения/записи в памяти пользовательского режима;

– AcquireSRWLockShared() — захват замка в совместном (неблокирующем других читателей) режиме для чтения массива;

– ReleaseSRWLockShared() — освобождение совместного замка потоком-читателем;

– AcquireSRWLockExclusive() — монопольный захват замка потоком-писателем с блокировкой всех остальных читателей и писателей;

– ReleaseSRWLockExclusive() — освобождение монопольного замка после завершения транзакции модификации;

– CreateThread() — создание и запуск независимых рабочих потоков читателей и писателей;

– WaitForMultipleObjects() — барьерная синхронизация главного потока до момента завершения работы всех участников эксперимента;

– CloseHandle() — закрытие системных дескрипторов отработавших потоков ядра;

– QueryPerformanceFrequency() — определение тактовой частоты высокоточного аппаратного счетчика времени;

– QueryPerformanceCounter() — измерение временных задержек нахождения читателей в очереди блокировки с точностью до микросекунд;

– SetConsoleCP() и SetConsoleOutputCP() — настройка кодовых страниц вывода терминала на UTF-8.

3.4 Основные фрагменты исходного кода

Программная реализация разделена на логические блоки, обеспечивающие генерацию транзакций писателями, проверку целостности данных читателями и управление жизненным циклом потоков.

В листинге 1 приведена функция рабочего потока писателя, демонстрирующая разделение на две стратегии синхронизации. В зависимости от конфигурации поток либо захватывает эксклюзивный замок AcquireSRWLockExclusive() на всю серию итераций, либо производит изолированный захват и освобождение замка на каждый атомарный перенос значения между ячейками массива.

```
DWORD WINAPI WriterThreadProc(LPVOID lpParam) {
    auto* args = static_cast<ThreadArgs*>(lpParam);
    auto* arr = args->sharedArray;
    std::mt19937 gen(SEED_BASE + args->threadId);
```

```
    std::uniform_int_distribution<int> indexDist(0, ARRAY_SIZE - 1);
    std::uniform_int_distribution<int> deltaDist(1, 5);

    int groupsCount = OPERATIONS_COUNT / args->config.groupSize;

    for (int g = 0; g < groupsCount; ++g) {
        if (args->config.strategy == LOCK_GROUP) {
            AcquireSRWLockExclusive(&arr->lock);
            for (int k = 0; k < args->config.groupSize; ++k) {
                int src = indexDist(gen), dst = indexDist(gen);
                if (src == dst) continue;
                int delta = deltaDist(gen);
                arr->data[src] -= delta;
                arr->data[dst] += delta;
            }
            ReleaseSRWLockExclusive(&arr->lock);
        } else {
            for (int k = 0; k < args->config.groupSize; ++k) {
                int src = indexDist(gen), dst = indexDist(gen);
                if (src == dst) continue;
                int delta = deltaDist(gen);
                AcquireSRWLockExclusive(&arr->lock);
                arr->data[src] -= delta;
                arr->data[dst] += delta;
                ReleaseSRWLockExclusive(&arr->lock);
            }
        }
    }
    return 0;
}
```

Листинг 1 – Поток писателя с поддержкой двух стратегий блокировки

Алгоритм функционирования читателя представлен в листинге 2. В цикле производится замер длительности ожидания захвата замка в совместном режиме AcquireSRWLockShared(), подсчет суммы 100 элементов и верификация инварианта целостности.

```
DWORD WINAPI ReaderThreadProc(LPVOID lpParam) {
    auto* args = static_cast<ThreadArgs*>(lpParam);
    auto* arr = args->sharedArray;
    PrecisionTimer waitTimer;

    for (int i = 0; i < OPERATIONS_COUNT; ++i) {
        waitTimer.Start();
        AcquireSRWLockShared(&arr->lock);
        waitTimer.Stop();
```

```
            args->stats.totalWaitTimeMs +=
(waitTimer.GetMicroseconds() / 1000.0);
            args->stats.totalReadOps++;

            int currentSum = 0;
            for (int j = 0; j < ARRAY_SIZE; ++j) {
                currentSum += arr->data[j];
            }

            if (currentSum != EXPECTED_SUM) {
                args->stats.invariantViolations++;
            }
            ReleaseSRWLockShared(&arr->lock);
        }
        return 0;
    }
```

Листинг 2 – Поток читателя с замером времени ожидания и контролем инварианта

3.5 Результаты экспериментального исследования

Экспериментальное исследование проводилось при стандартной конфигурации из трех читателей и двух писателей, каждый из которых выполнял по 560 операций в соответствии с индивидуальным параметром STUDENT_ID = 72. Было исследовано четыре сценария работы: комбинации групп по 5 и 20 операций при групповой и поодиночной блокировках.

Сводные результаты экспериментальных измерений представлены на рисунке 2.

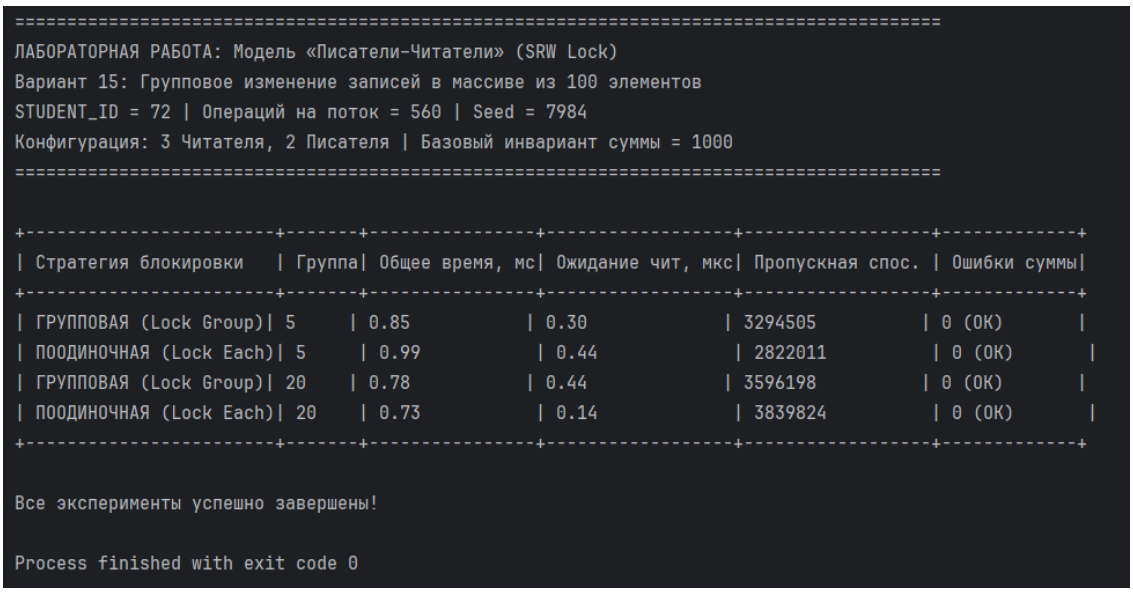


Рисунок 2 – Результаты сравнительного анализа стратегий блокировки SRWLock

Анализ экспериментальных данных подтверждает стопроцентную корректность работы механизма синхронизации: во всех четырех конфигурациях количество нарушений инварианта суммы составило строго 0, что гарантирует отсутствие гонок данных при одновременных модификациях массива. Общая пропускная способность системы во всех режимах превысила 2,8 миллиона операций в секунду, подтверждая исключительную эффективность легковесного примитива SRWLOCK за счет минимизации переходов между режимами ядра и пользователя.

Сопоставление стратегий блокировки наглядно демонстрирует фундаментальный компромисс между задержкой читателей и гранулярностью блокировок. При размере группы в 20 операций переход от групповой блокировки к поодиночной привел к трехкратному падению среднего времени ожидания читателей в очереди – с 0,44 мкс до рекордных 0,14 мкс. Это объясняется тем, что при частых коротких блокировках между отдельными переносами возникают временные интервалы, в которые читатели успевают беспрепятственно осуществить совместный захват ресурса, не простаивая в ожидании завершения всей пачки из 20 транзакций.

3.6 Журнал разработки и контроль версий

В соответствии с требованиями раздела 7.3 методических указаний процесс создания программного комплекса был разбит на четыре логических

этапа с обязательной фиксацией промежуточных результатов в системе контроля версий Git.

На первом этапе был создан проект в среде CLion, разработаны структуры данных общего массива на 100 элементов, внедрен примитив синхронизации SRWLOCK с вызовом InitializeSRWLock() и подготовлен класс высокоточного таймера для замера задержек в микросекундах .

Второй этап был посвящен созданию рабочих потоков средствами Windows API: были реализованы функции WriterThreadProc() и ReaderThreadProc(), интегрирован потокобезопасный генератор псевдослучайных чисел std::mt19937 с индивидуальным зерном 7984 и организован базовый перенос значений между ячейками массива.

На третьем этапе была реализована ключевая логика разделения стратегий блокировки: введены сценарии групповой блокировки всей пачки операций и поодиночной блокировки каждого элементарного действия, а также добавлен механизм непрерывного контроля математического инварианта сохранения суммы 1000.

Финальный четвертый этап включал объединение экспериментов в файле main.cpp, проведение замеров производительности для групп из 5 и 20 операций при конфигурации из трех читателей и двух писателей, а также форматированный вывод сводной таблицы с показателями времени ожидания и пропускной способности.

4 ИСПОЛЬЗОВАНИЕ ИИ

В соответствии с разделом 10 методических указаний при подготовке и выполнении лабораторной работы применялась большая языковая модель Gemini. Инструмент привлекался в качестве справочной системы для консультаций по узким теоретическим вопросам системного программирования.

В частности, с помощью модели были детально исследованы различия между примитивами SRWLOCK, CRITICAL_SECTION и Mutex, проанализированы правила предотвращения взаимных блокировок при конкурентных модификациях нескольких ячеек памяти, а также рассмотрены алгоритмические подходы к высокоточному измерению времени ожидания потоков в очереди с помощью аппаратного таймера QueryPerformanceCounter(). Все структуры данных, генераторы случайных чисел с индивидуальным параметром SEED = 7984, логика двух стратегий блокировки писателей, код потоков на C++20 в среде CLion и журнал Git-коммитов были разработаны, протестированы и верифицированы студентом лично. Программный код понятен в полном объеме.

ВЫВОД

В ходе выполнения лабораторной работы были изучены современные низкоуровневые средства межпоточной синхронизации в операционной системе Windows. На практике была успешно решена классическая проблема конкурентного доступа «Писатели–Читатели» с использованием легковесного системного примитива SRWLOCK (Slim Reader/Writer Lock). Было разработано многопоточное приложение, моделирующее параллельное перемещение числовых значений внутри разделяемого массива из 100 элементов пулом из трех читателей и двух писателей с сохранением базового математического инварианта общей суммы, равной 1000.

Экспериментальное исследование полностью подтвердило корректность выбранной схемы синхронизации: во всех тестовых испытаниях число зафиксированных нарушений инварианта суммы составило строго ноль, что свидетельствует о полном отсутствии состояний гонки (Race Condition) и гарантирует строгую согласованность данных. Использование механизма SRWLOCK обеспечило исключительно высокую пропускную способность, превысившую во всех сценариях 2,8 миллиона операций в секунду, что объясняется эффективной работой примитива в пользовательском режиме (User Mode) на базе быстрых атомарных инструкций процессора без создания тяжелых дескрипторов ядра.

Сравнительный анализ гранулярности блокировок выявил ключевую зависимость между задержками читателей и производительностью писателей. При увеличении размера группы до 20 операций переход от групповой блокировки к поодиночной привел к трехкратному сокращению среднего времени ожидания читателей в очереди – с 0,44 мкс до 0,14 мкс при достижении пиковой пропускной способности свыше 3,8 миллиона операций в секунду. Это доказывает, что разбиение длительной монопольной блокировки на серию коротких атомарных транзакций открывает временные окна для параллельного совместного доступа читателей (AcquireSRWLockShared), исключая эффект «голодания» читающих потоков без деградации общей скорости обработки данных.